

## Practical 11

To use JavaScript **timers** (setTimeout() and setInterval()), implement **dynamic UI updates**, and organize code using **ES6 modules (import/export)** to create a countdown or class scheduler that highlights active sessions and adapts to viewport changes.

### Requirements

#### Software / Tools

- Web Browser
- Code Editor
- Local server setup (VS Code Live Server or Node.js) — required for ES6 modules

#### Files

- index.html
- main.js (main script file)
- timer.js (module file)
- ui.js (module file)

### Theory

#### 1. Timers in JavaScript

JavaScript provides **built-in timer functions** that help execute code **after a delay** or **repeatedly at intervals**.

| Method                                              | Description                                                                                                               | Example                                                                                                  |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| setTimeout(fn,<br><br>delay)<br><br>setInterval(fn, | Executes fn <b>once</b> after the given delay (in ms).<br><br>Executes fn <b>repeatedly</b> at every given delay (in ms). | setTimeout(() =><br>console.log("Hello"), 2000);<br><br>setInterval(() =><br>console.log("Tick"), 1000); |

|                                     |                                                  |                         |
|-------------------------------------|--------------------------------------------------|-------------------------|
| delay)                              |                                                  |                         |
| clearTimeout() /<br>clearInterval() | Stops the timer created<br>by the above methods. | clearInterval(timerId); |

## 2. Dynamic UI

Dynamic UI refers to **real-time updates** of webpage content using JavaScript — such as:

- Updating time every second.
- Highlighting the active schedule session.
- Adjusting styles or layouts when the window is resized.

## 3. Event Delegation

Event delegation means attaching a **single event listener** to a **parent element** instead of each child. It uses the concept of **event bubbling**.

Example:

```
document.getElementById('list').addEventListener('click', (e) => {
  if (e.target.tagName === 'LI') {
    alert("You clicked " + e.target.textContent);
  }
});
```

## 4. ES6 Modules

Modules allow you to **split code into separate files** and **reuse** it easily.

| Keyword | Purpose                                                     |
|---------|-------------------------------------------------------------|
| export  | Makes functions/variables available for use in other files. |
| import  | Brings in code from another module.                         |

Example:

```
// math.js
export function add(a, b) { return a + b; }
```

```
// main.js
import { add } from './math.js';
console.log(add(2, 3));
```

**Note:** You must run ES6 modules using a **server environment** — not just by opening HTML directly.

### Steps

1. Create a folder Practical11\_Timers\_Modules.
2. Inside it, create these files:
  - index.html
  - main.js
  - timer.js
  - ui.js
3. Add a basic HTML structure and load the main script using type="module".
4. Implement timer and UI functions in separate modules.
5. Import them in main.js and call the functions.
6. [Test in browser using VS Code Live Server or any local server.](#)

### Example Implementation

#### index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Timers, Dynamic UI & Modular Design</title>

  <!-- Basic styling for the page -->
  <style>
    body { font-family: Arial; padding: 20px; }
```

```

/* Each class session block */
.session {
  padding: 10px;
  margin: 5px 0;
  border: 1px solid #ccc;
  border-radius: 5px;
}

/* Highlight active session */
.active {
  background: #c8f7c5;
}
</style>
</head>
<body>
  <h2>Class Scheduler (with Timers & Modules)</h2>

  <!-- Container for all sessions -->
  <div id="sessions">
    <div class="session" data-time="09:00">HTML Basics - 9:00 AM</div>
    <div class="session" data-time="10:00">CSS Design - 10:00 AM</div> <div
    class="session" data-time="11:00">JavaScript Logic - 11:00 AM</div>
  </div>

  <!-- Display countdown timer -->
  <p id="countdown"></p>

  <!-- Main entry JavaScript file (module-based) -->
  <script type="module" src="main.js"></script>
</body>
</html>

```

**timer.js**

```

// timer.js — contains countdown and session highlight logic

// Function to start a countdown timer
export function startCountdown(duration, display) {
  let time = duration; // time in seconds

  // setInterval() runs the code every 1 second
  const timer = setInterval(() => {
    const minutes = Math.floor(time / 60); // convert seconds to minutes
    const seconds = time % 60; // remaining seconds

    // Update countdown message on the page
    display.textContent = `Next session starts in ${minutes}:${seconds < 10 ? '0' + seconds
: seconds}`;

    time--; // decrease the timer each second

    // Stop timer when it reaches 0
    if (time < 0) {
      clearInterval(timer);
      display.textContent = "Session Started!";
    }
  }, 1000); // executes every 1000 milliseconds (1 second)
}

// Function to highlight the currently active session
export function highlightActiveSession() {
  const sessions = document.querySelectorAll('.session'); // get all session elements
  const now = new Date(); // get current date/time
  const currentHour = now.getHours(); // extract current hour (24-hour format)

  // Loop through all session divs
  sessions.forEach(session => {
    const hour = parseInt(session.dataset.time.split(':')[0]); // extract hour from data-time
  });
}

```

```

// If current hour matches session time → highlight
if (hour === currentHour) {
  session.classList.add('active');
} else {
  session.classList.remove('active');
}
});
}

```

### **ui.js**

```

// ui.js — handles dynamic UI and responsive design behavior
// Function to change background color based on screen size
export function handleViewportChange() {
  // If screen width < 600px, apply a light blue background
  if (window.innerWidth < 600) {
    document.body.style.background = '#f0f8ff';
  } else {
    // For larger screens, use white background
    document.body.style.background = 'white';
  }
}

```

```

// Function to add dynamic behavior (event delegation)
export function addDynamicSessions() {
  const sessionsDiv = document.getElementById('sessions');

  // Add event listener to parent container (event delegation)
  sessionsDiv.addEventListener('click', (e) => {
    // Check if clicked element has class "session"
    if (e.target.classList.contains('session')) {
      alert("You selected: " + e.target.textContent);
    }
  })
}

```

```
});  
}
```

### **main.js**

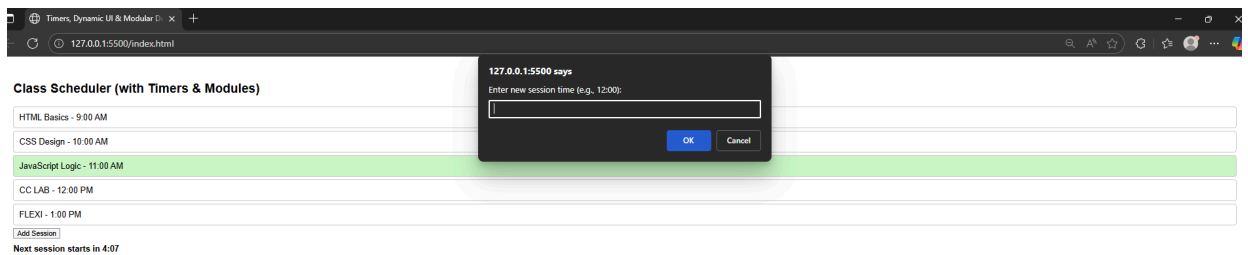
```
// Import functions from timer.js and ui.js modules  
import { startCountdown, highlightActiveSession } from './timer.js';  
import { handleViewportChange, addDynamicSessions } from './ui.js';  
  
// Select the countdown display element from the DOM const  
countdownDisplay = document.getElementById('countdown');  
  
// Start countdown for next class (e.g., 5 minutes = 300 seconds)  
startCountdown(300, countdownDisplay);  
  
// Highlight the active session every minute automatically  
setInterval(highlightActiveSession, 60000); // recheck every 60 seconds  
highlightActiveSession(); // run once immediately on load  
  
// Handle screen resizing for responsive behavior  
window.addEventListener('resize', handleViewportChange);  
handleViewportChange(); // run on page load as well  
  
// Enable event delegation on session elements  
addDynamicSessions();
```

### **Output**

- Displays **class schedule**.
- Shows a **countdown timer** (updates every second).
- Automatically **highlights the current class session** based on system time.
- Background color **changes** when window is resized (responsive UI). ●
- Clicking a session triggers an **alert** (event delegation).

## Task

1. Add a **new session dynamically** using a button (“Add Session”).
2. Modify the countdown so that it changes color when time  $< 1$  minute.
3. Use a separate module helper.js to format time or log events.
4. Add responsiveness — change font size when width  $< 500\text{px}$ .



TIME REMAINING  $< 1$  min:-

